Chủ đề 8: Kết nối Internet

Nội dung bài học

- Bài giảng này sẽ cung cấp kiến thức và kỹ năng về:
- Tổng quan về kết nối mạng trên Android.
- Các giao thức và phương thức kết nối phổ biến.
- Các dịch vụ Web: SOAP vs RESTful API.
- Cách triển khai RESTful API trên Android.
- Sử dụng các thư viện hỗ trợ kết nối mạng như Retrofit.
- Ứng dụng Coroutines để xử lý bất đồng bộ.
- Firebase và khả năng kết nối thời gian thực.
- Bài tập thực hành và thảo luận để củng cố kiến thức.

Chủ đề 8: Kết nối Internet

Mục tiêu

- Sau khi hoàn thành bài học, sinh viên có thể:
- Hiểu được vai trò và tầm quan trọng của kết nối mạng trong ứng dụng di động.
- Phân biệt được các giao thức mạng và lựa chọn phương pháp kết nối phù hợp.
- Áp dụng Retrofit để giao tiếp với RESTful API.
- Sử dụng Coroutines để tối ưu xử lý bất đồng bộ khi kết nối mạng.
- Triển khai Firebase để hỗ trợ kết nối dữ liệu thời gian thực.
- Viết các ứng dụng có khả năng giao tiếp mạng hiệu quả, bảo mật và tối ưu.

Giới thiệu về lập trình kết nối mạng

- **Tầm quan trọng của kết nối mạng trên Android:**

  - Ứng dụng lấy dữ liệu từ server.

  - Cung cấp trải nghiệm theo thời gian thực cho người dùng.

- **Các ứng dụng phổ biến:**

  - Ứng dụng tin tức.

  - Ứng dụng thương mại điện tử.

  - Ứng dụng mạng xã hội.

Các giao thức và kết nối phổ biến

- **Các giao thức mạng:**

  - HTTP/HTTPS: Giao thức nền tảng cho kết nối web.

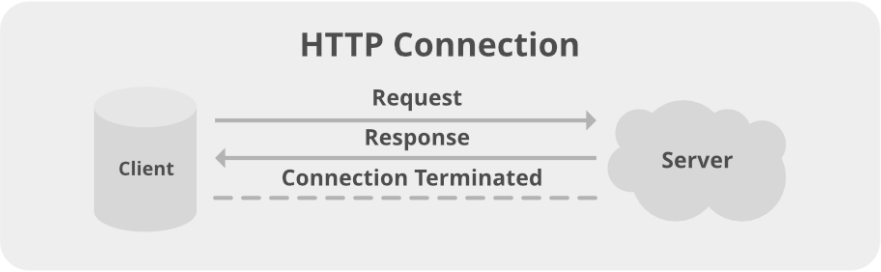
  - WebSocket: Hỗ trợ giao tiếp thời gian thực.

- **Các kết nối phổ biến:**

  - Wi-Fi: Kết nối không dây .

  - Bluetooth: Giao tiếp giữa các thiết bị cận kề.

  - NFC: Kết nối tốc độ cao với phạm vi rất gần.



Các loại dịch vụ web: XML/SOAP vs RESTful

- **XML/SOAP:**

  - Định dạng dữ liệu: XML.

  - Tính năng: Hỗ trợ bảo mật và giao dịch phức tạp.

  - Nhược điểm: Cồng kềnh, nặng hơn RESTful.

- **RESTful:**

  - Định dạng dữ liệu: JSON hoặc các định dạng khác.

  - Tính năng: Nhẹ, dễ triển khai.

  - Nhược điểm: Không hỗ trợ đầy đủ các tính năng bảo mật nâng cao như SOAP.

Giới thiệu RESTful và JSON

- **RESTful:**

  - Giao tiếp client-server qua HTTP.

  - Dựa trên các phương thức HTTP: GET, POST, PUT, DELETE.

- **JSON:** Định dạng dữ liệu nhẹ và dễ đọc.

```
{
"id": 1,
"name": "John Doe",
"email": "john@example.com"
}
```

Ưu và nhược điểm của RESTful

- **Ưu điểm:**

  - Nhẹ và hiệu quả.

  - Tích hợp dễ dàng với hầu hết các hệ thống.

  - Được hỗ trợ rộng rãi bởi cộng đồng.

- **Nhược điểm:**

  - Không hỗ trợ các tính năng giao dịch phức tạp.

  - Yêu cầu tự xây dựng các cơ chế bảo mật bổ sung.

Các phương thức HTTP: GET và POST

- **GET:**

  - Lấy dữ liệu từ server.

  - Không chứa dữ liệu trong body của yêu cầu.

  - Sử dụng trong các ứng dụng đọc dữ liệu.

- **POST:**

  - Gửi dữ liệu đến server để xử lý.

  - Sử dụng để tạo mới hoặc gửi biểu mẫu.

  - Có thể gửi dữ liệu trong body của yêu cầu.

Ví dụ sử dụng phương thức GET

- **Mã mẫu:**

```
val url = URL("https://example.com/api")
val connection = url.openConnection() as HttpURLConnection
connection.requestMethod = "GET"
val response = connection.inputStream.bufferedReader().use { it.readText() }
println(response)
```

- **Quy trình:**

1. Khởi tạo URL và mở kết nối.
2. Đặt phương thức GET.
3. Đọc phản hồi từ server.

Ví dụ sử dụng phương thức POST

- **Mã mẫu:**

```
val url = URL("https://example.com/api")
val connection = url.openConnection() as HttpURLConnection
connection.requestMethod = "POST"
connection.doOutput = true
connection.outputStream.write("data".toByteArray())

val response = connection.inputStream.bufferedReader().use { it.readText() }
println(response)
```

Ví dụ sử dụng phương thức POST

- **Quy trình:**

1. Khởi tạo URL và mở kết nối.
2. Đặt phương thức POST.
3. Gửi dữ liệu qua outputStream.
4. Đọc phản hồi từ server.

Các thư viện hỗ trợ kết nối mang

HttpURLConnection

Nhẹ, có sẵn trong Android.

Yêu cầu nhiều mã nguồn để xử lý lỗi và đóng kết nối.

Apache HttpClient

Hỗ trợ HTTP cấp cao.

Không còn được tích hợp từ Android 6.0.

Retrofit

Tích hợp tốt với RESTful API.

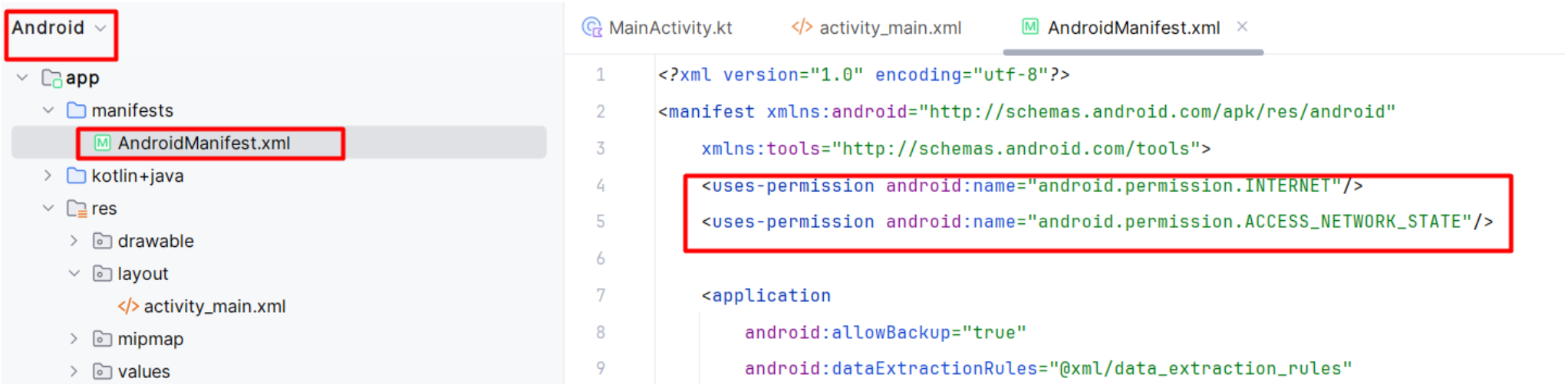
Dễ sử dụng, hỗ trợ Gson và Kotlin Coroutines.

HttpURLConnection - Thêm quyền truy cập mạng

- Để sử dụng HttpURLConnection, cần thêm quyền truy cập mạng trong AndroidManifest.xml:

<uses-permission android:name="android.permission.INTERNET"/>

- Nếu thiếu quyền này, ứng dụng sẽ không thể kết nối mạng.



HttpURLConnection - Khởi tạo kết nối

- **Cách khởi tạo kết nối:**

```
val url = URL("https://example.com/api")
val connection = url.openConnection() as HttpURLConnection
connection.requestMethod = "GET"
```

- **Lưu ý:**

- URL: Chứa endpoint của API.

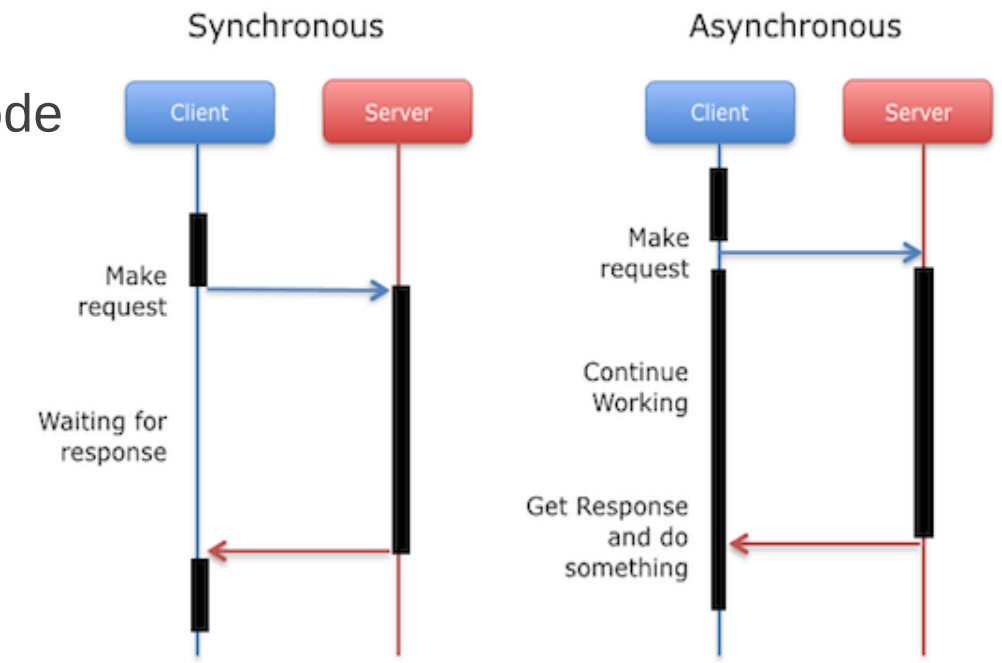
- openConnection(): Tạo kết nối với server.

- requestMethod: Đặt phương thức HTTP (GET/POST).

HttpURLConnection - Gửi yêu cầu

- **Gửi yêu cầu:**

```
connection.connect()
val responseCode = connection.responseCode
println("Response Code: $responseCode")
```



HttpURLConnection - Đọc dữ liệu phản hồi

- **Đọc dữ liệu từ server:**

```
val inputStream = connection.inputStream
val response = inputStream.bufferedReader().use { it.readText() }
println(response)
```

- **Lưu ý:**

- Sử dụng bufferedReader() để đọc nội dung phản hồi từ server.

- Đóng inputStream sau khi đọc xong.

HttpURLConnection - Đóng kết nối

- Sau khi hoàn tất xử lý, đóng kết nối để giải phóng tài nguyên:

  connection.disconnect()

- **Lưu ý:**

- Đóng kết nối giúp tránh rò rỉ tài nguyên.

- Luôn đóng kết nối trong các ứng dụng thực tế.

Retrofit - Giới thiệu

- Thư viện mạnh mẽ hỗ trợ kết nối RESTful API.
  - Tự động chuyển đổi JSON sang đối tượng Kotlin.
  - Hỗ trợ xử lý đồng bộ và bất đồng bộ.

Retrofit - Thêm phụ thuộc

- Thêm vào tệp build.gradle:

```
implementation "com.squareup.retrofit2:retrofit:2.9.0"

implementation "com.squareup.retrofit2:converter-gson:2.9.0"
```

- retrofit: Thư viện chính cho kết nối RESTful.
- converter-gson: Chuyển đổi JSON sang đối tượng Kotlin.

Retrofit - Khởi tạo Retrofit

- Cách cấu hình Retrofit:

```
val retrofit = Retrofit.Builder()
                .baseUrl("https://example.com/")
                .addConverterFactory(GsonConverterFactory.create())
                .build()
```

- baseUrl: Địa chỉ gốc của API.
- addConverterFactory: Chuyển đổi dữ liệu JSON sang đối tượng Kotlin.

Retrofit - Định nghĩa interface API

- Tạo interface để khai báo endpoint API:

```
interface ApiService {
  @GET("users")
  suspend fun getUsers(): List<User>
}
```

- @GET: Gửi yêu cầu GET đến endpoint users.
- suspend fun: Hỗ trợ Kotlin Coroutines cho xử lý bất đồng bộ.

Retrofit - Kết nối với API

- Kết nối interface với Retrofit:

```
val apiService = retrofit.create(ApiService::class.java)
val users = apiService.getUsers()
println(users)
```

- Sử dụng retrofit.create() để tạo instance của interface API.

Retrofit - Thêm các phương thức khác

- Ví dụ sử dụng POST request:

```
@POST("users")
suspend fun createUser(@Body user: User): Response<User>
```

- @POST: Gửi yêu cầu POST đến endpoint users.
- @Body: Gửi dữ liệu trong body của yêu cầu.

Retrofit - Ưu và nhược điểm

- Ưu điểm:
  - Dễ dàng sử dụng với RESTful API.
  - Tích hợp tốt với Gson và Kotlin Coroutines.
- Nhược điểm:
  - Phụ thuộc vào thư viện bên ngoài.
  - Cấu hình phức tạp hơn cho các trường hợp đặc biệt.

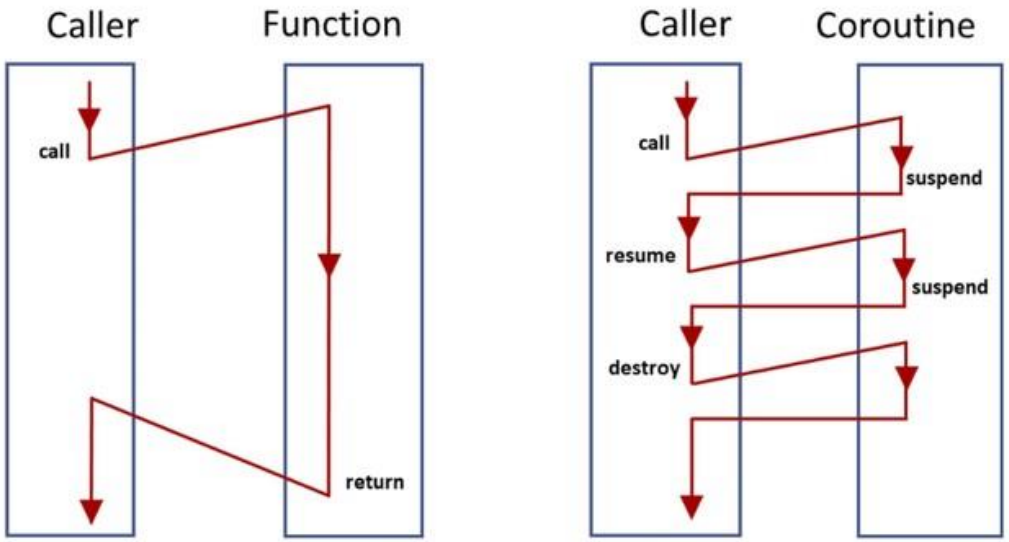
Sử dụng Coroutines trong kết nối mạng

Giới thiệu về Coroutines

Coroutines giúp xử lý tác vụ bất đồng bộ một cách hiệu quả.

Giảm độ phức tạp so với việc sử dụng Thread truyền thống.

Hoạt động nhẹ hơn so với Thread thông thường.



Sử dụng Coroutines trong kết nối mạng

Các thành phần chính của Coroutines

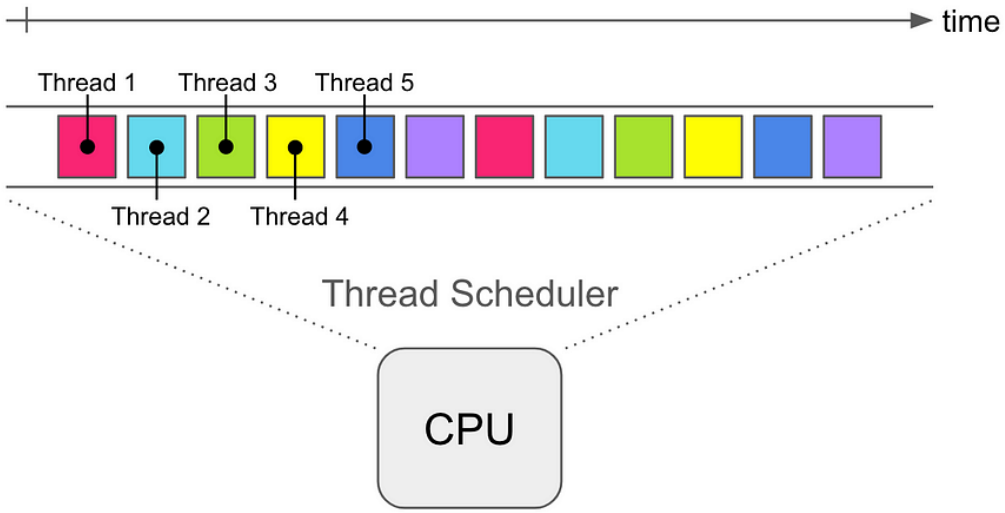
CoroutineScope: Phạm vi hoạt động của

Coroutines.Dispatchers: Xác định luồng thực thi:

- Dispatchers.Main: Chạy trên UI thread.
- Dispatchers.IO: Dành cho tác vụ I/O như gọi API.
- Dispatchers.Default: Dành cho tác vụ tính toán.

Suspend functions: Hàm có thể chạy bất đồng bộ.

Jobs & Deferred: Quản lý tiến trình Coroutines.

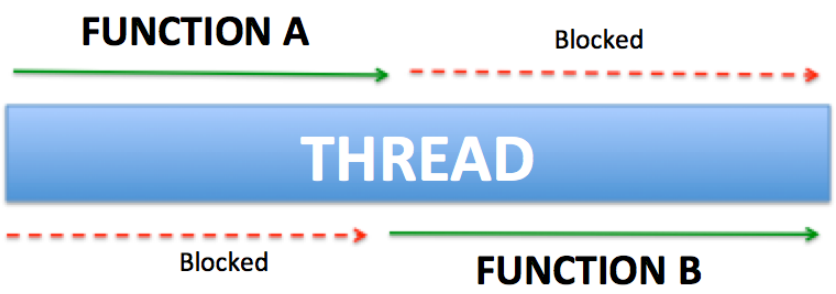


Sử dụng Coroutines trong kết nối mạng

Kết hợp Coroutines với Retrofit

Sử dụng suspend fun trong Retrofit API:

```
interface ApiService {
    @GET("users")
    suspend fun getUsers(): List<User>
}
```



Sử dụng Coroutines trong kết nối mạng

Ví dụ sử dụng Coroutines với Retrofit

Dispatchers.IO: Dùng để xử lý công việc nặng về I/O.

withContext(Dispatchers.Main): Đưa kết quả về luồng chính.

```
val apiService = retrofit.create(ApiService::class.java)

CoroutineScope(Dispatchers.IO).launch {
    val users = apiService.getUsers()
    withContext(Dispatchers.Main) {
        println(users)
    }
}
```

Sử dụng Coroutines trong kết nối mạng

So sánh Coroutines với Thread

Tiêu chí	Coroutines	Thread
Quản lý	Dễ dàng với `CoroutineScope`	Khó khăn hơn với `ThreadPool`
Hiệu suất	Nhẹ và nhanh hơn	Tốn tài nguyên hơn
Xử lý đồng bộ	Hỗ trợ tốt với `suspend fun`	Cần xử lý thủ công

So sánh các thư viện

- **HttpURLConnection:**

  - Nhẹ, có sẵn trong Android.

  - Yêu cầu nhiều mã hơn để xử lý lỗi và đóng kết nối.

- **Apache HttpClient:**

  - Hỗ trợ HTTP mức cao.

  - Không còn được tích hợp từ Android 6.0.

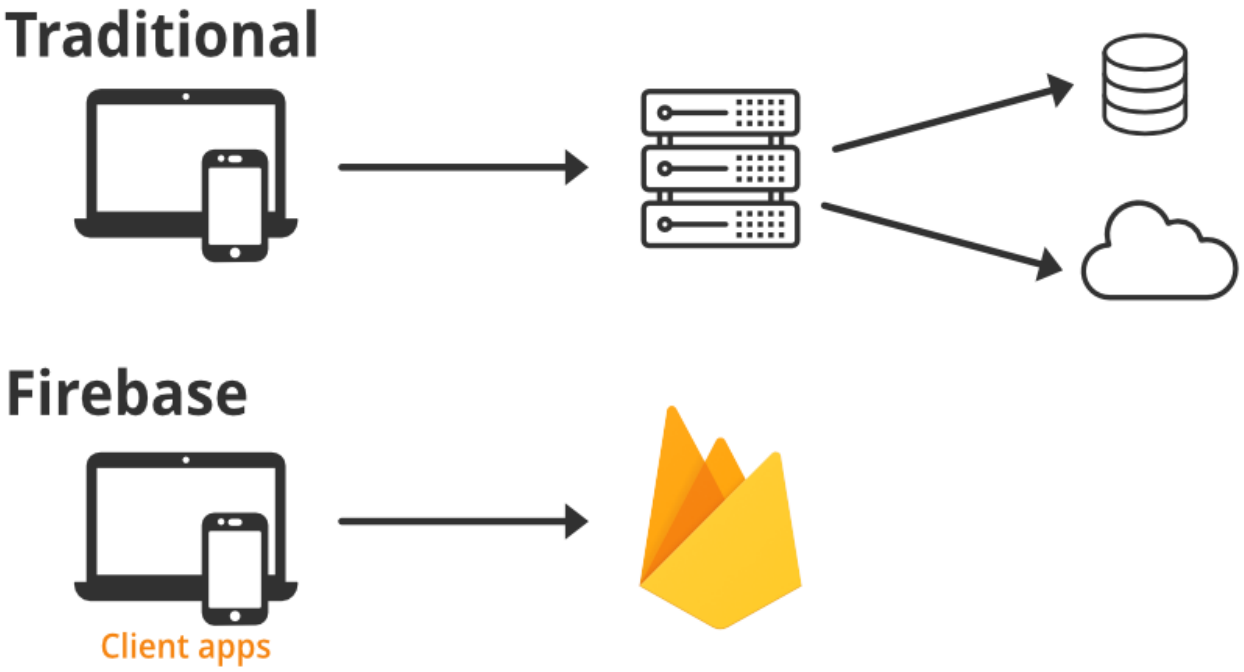
- **Retrofit:**

  - Tích hợp tốt với RESTful API.

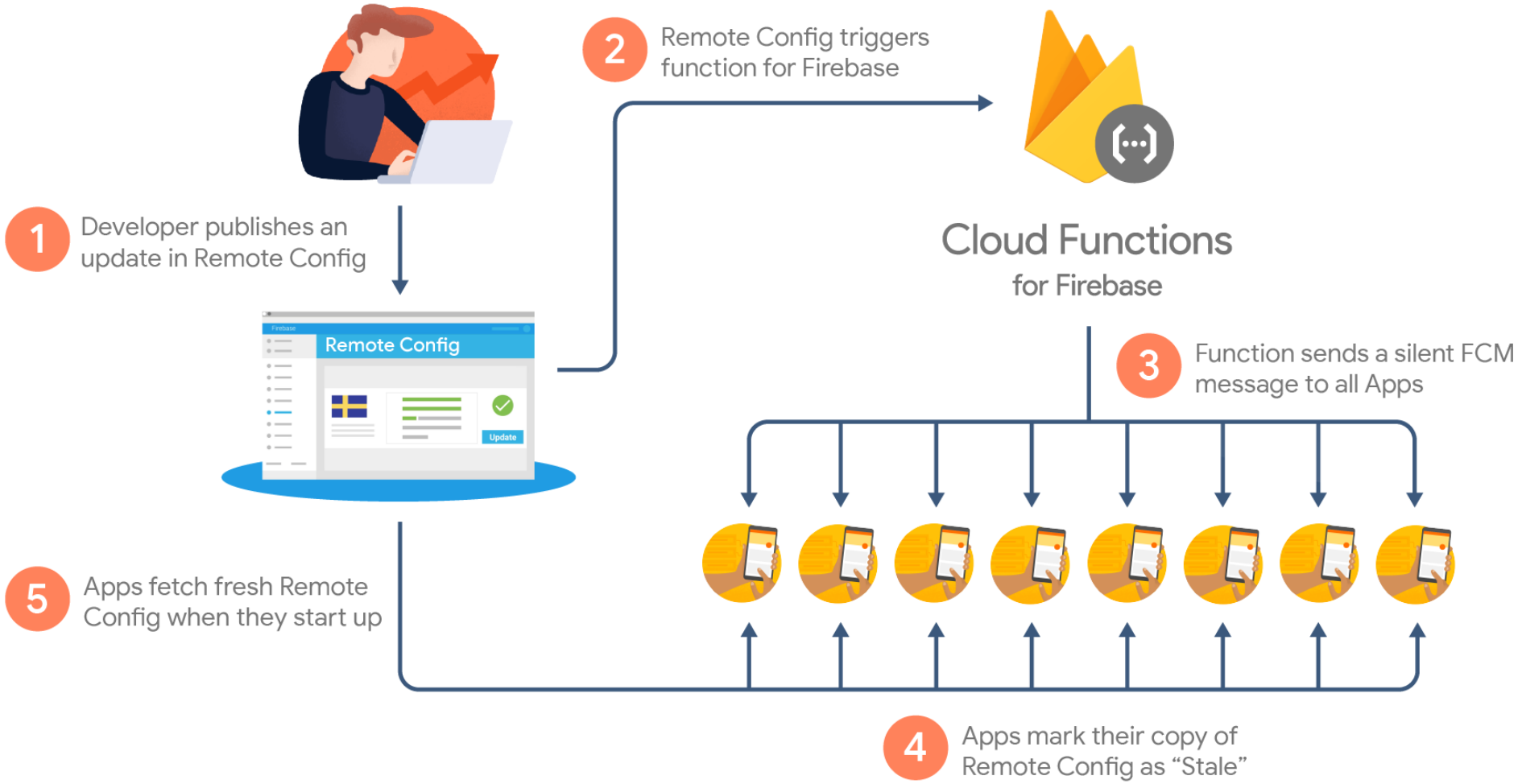
  - Hỗ trợ mạnh mẽ, dễ sử dụng, phổ biến hơn.

Firebase - Giới thiệu

- Firebase là nền tảng phát triển ứng dụng di động và web của Google.
- Các tính năng chính:
  - Realtime Database
  - Firestore
  - Firebase Authentication
  - Cloud Storage
  - Firebase Analytics



Firebase - Realtime Database



Firebase - Realtime Database

- **Cơ sở dữ liệu thời gian thực:**

  - Lưu trữ dữ liệu dạng JSON.

  - Tự động đồng bộ hóa dữ liệu trên thiết bị.

- **Ví dụ mã:**

```
val database = Firebase.database.reference

  database.child("users").child("user1").setValue("Hello Firebase!")
```

Firebase - Firestore

- **Cloud Firestore:**

  - Cơ sở dữ liệu NoSQL.

  - Hỗ trợ truy vấn mạnh mẽ với các bộ lọc.

  - Tích hợp tính năng offline caching.

- **Ví dụ mã:**

```
val db = Firebase.firestore

  val user = hashMapOf("name" to "John", "email" to "john@example.com")

  db.collection("users").add(user)
```

Firebase - Authentication

- **Firebase Authentication hỗ trợ xác thực người dùng:**
  - Email/Password
  - Google, Facebook, Twitter
- **Ví dụ mã:**

```
Firebase.auth.createUserWithEmailAndPassword(email, password)
    .addOnCompleteListener { task ->
        if (task.isSuccessful) {
            val user = Firebase.auth.currentUser
            println("User created: ${user?.email}")
        }
    }
```

Firebase - Cloud Storage

- **Cloud Storage:**

  - Lưu trữ tệp tin như hình ảnh, video trên đám mây.

  - Bảo mật tích hợp với Firebase Authentication.

- **Ví dụ mã:**

```
val storage = Firebase.storage.reference

val file = Uri.fromFile(File("path/to/file"))

val uploadTask = storage.child("uploads/file.jpg").putFile(file)
```

Firebase - Tích hợp vào ứng dụng

- **Các bước tích hợp Firebase:**

1. Kết nối qua Firebase Console và thêm tệp google-services.json vào dự án.
2. Thêm phụ thuộc trong build.gradle:

```
implementation 'com.google.firebase:firebase-database-ktx'
implementation 'com.google.firebase:firebase-auth-ktx'
implementation 'com.google.firebase:firebase-firestore-ktx'
```

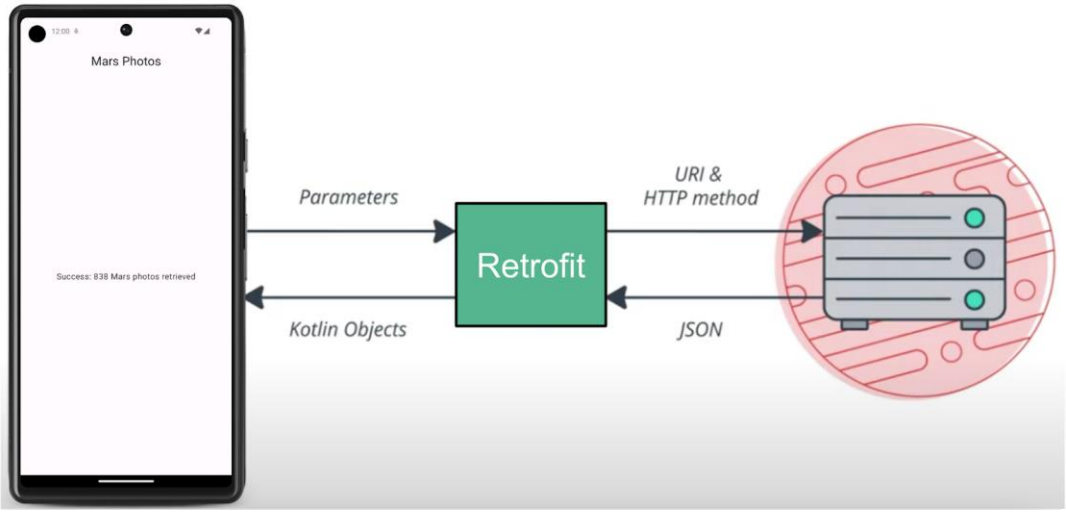
3. Sử dụng API Firebase theo tính năng cần thiết.

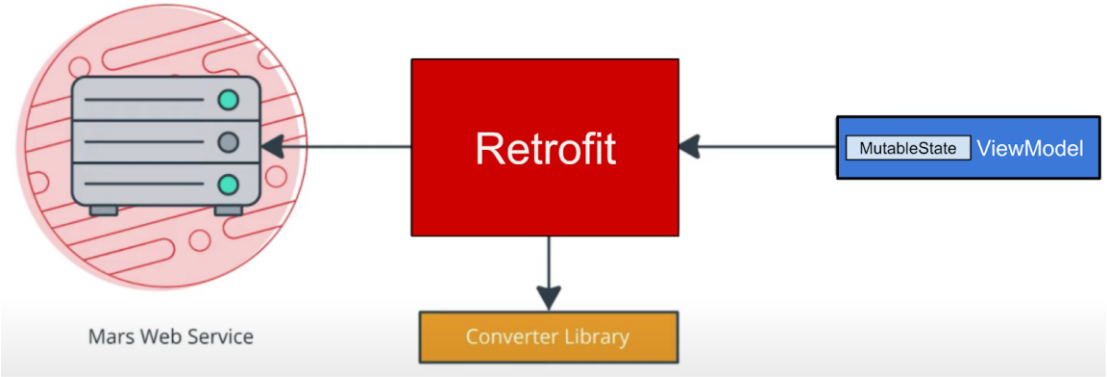
Thực hành: ứng dụng download ảnh sao hỏa

Ứng dụng demo lấy ảnh từ server và hiển thị trên điện thoại

Thực hành: ứng dụng download ảnh sao hỏa

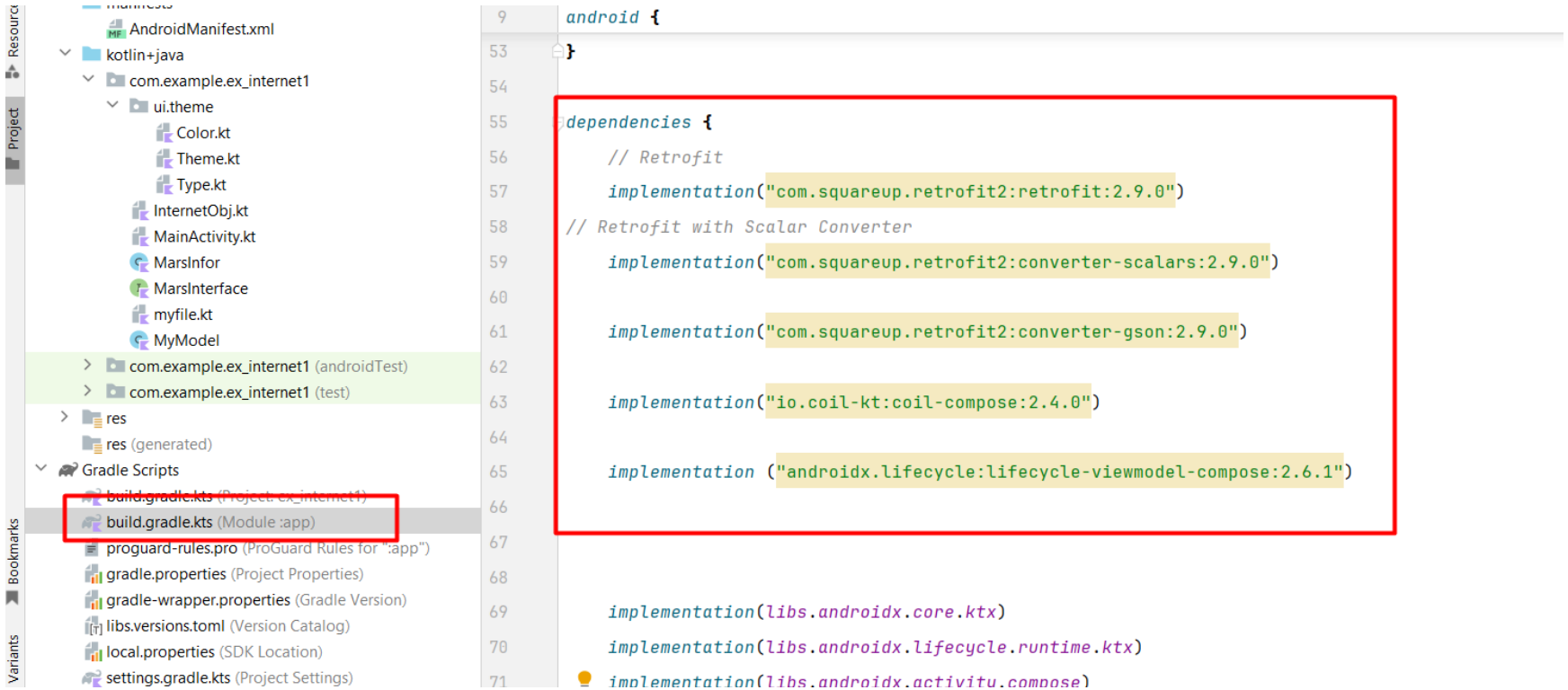
Ứng dụng demo lấy ảnh từ server và hiển thị trên điện thoại





Thực hành: ứng dụng download ảnh sao hỏa

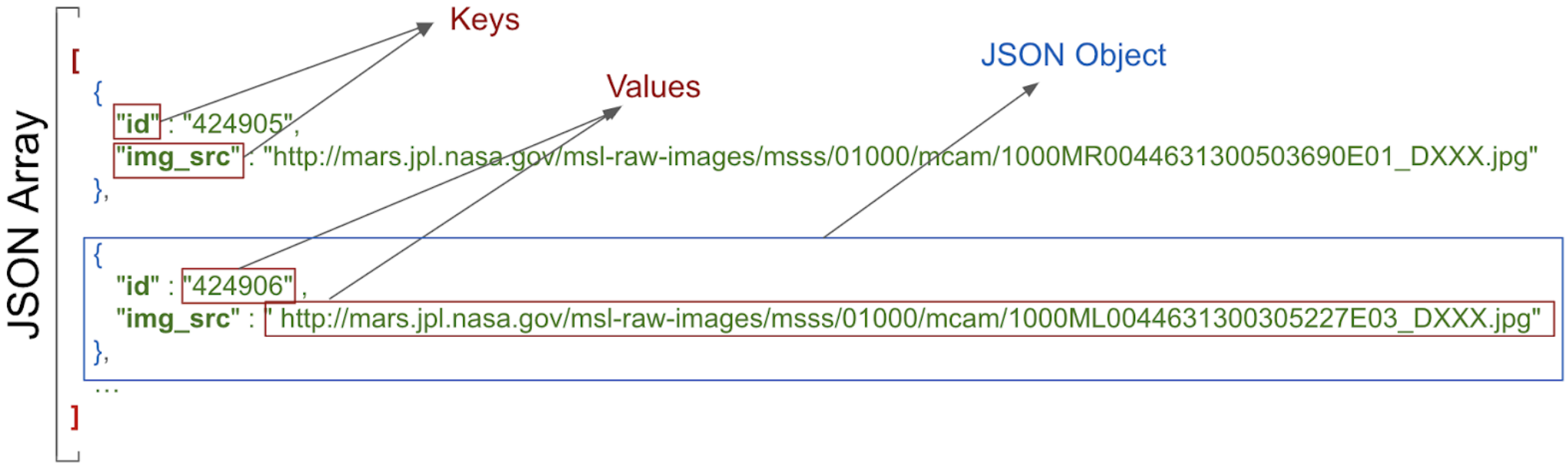
Bước 1: Cấu hình hệ thống



Thực hành: ứng dụng download ảnh sao hỏa

Bước 2: Tạo một lớp để lưu dữ liệu lấy về từ dịch vụ web (lớp kotlin)

```
data class Photo(    val id: String,    @SerializedName("img_src") val
image: String)
```

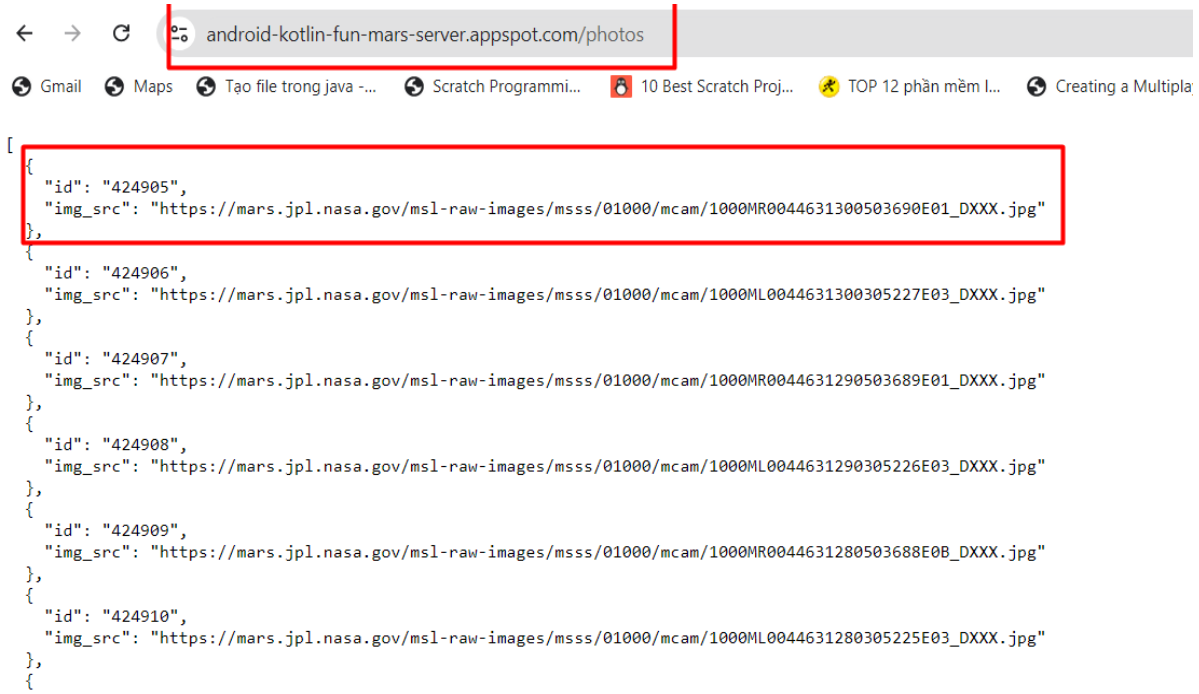


Thực hành: ứng dụng download ảnh sao hỏa

Bước 2: Tạo một lớp để lưu dữ liệu lấy về từ dịch vụ web (lớp kotlin)

```
data class Photo(    val id: String,    @SerializedName("img_src") val image: String)
```

annotation @SerializedName mô tả trường này tương đương với dữ liệu có tên img_src trên server:https://android-kotlin-fun-mars-server.appspot.com/photos



Thực hành: ứng dụng download ảnh sao hỏa

Bước 3: Tạo Interface

interface này sẽ cung cấp các hàm để làm việc với các dịch vụ của web, người lập trình chỉ cần mô tả phương thức, kiểu trả về còn những việc khác Retrofit sẽ xử lý: (interface trong thư mục kava+kotlin)

phương thức get với đường dẫn dịch vụ là photos : //https://android-kotlin-fun-mars-server.appspot.com/photos

```
import retrofit2.http.GET

interface MyPhoToService {
    @GET("photos") //
    suspend fun getPhoto() : List<Photo>
}
```

Thực hành: ứng dụng download ảnh sao hỏa

Bước 4: Tạo 1 đối tượng khởi tạo 1 lần để tạo kết nối đến server (chỉ tạo 1 đối tượng duy nhất)

```
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object myBuilder{
    fun create(): Retrofit{
        val retrofit = Retrofit.Builder()
            .baseUrl("https://android-kotlin-fun-mars-server.appspot.com/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
        return retrofit
    }
}
```

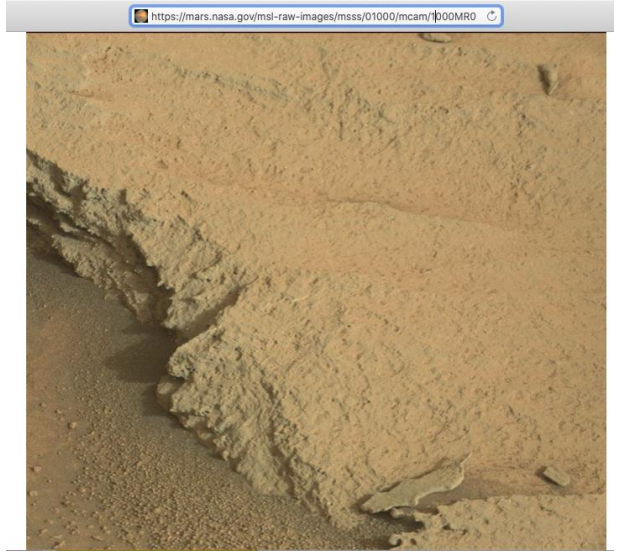
Thực hành: ứng dụng download ảnh sao hỏa

Bước 5: Kết nối và trao đổi dữ liệu, dưới đây là 1 đoạn chương trình đơn giản:

```
class MainActivity : AppCompatActivity() {
    lateinit var imageView: ImageView
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        imageView = findViewById(R.id.iv)
        GlobalScope.launch {
            val connect =
myBuilder.create().create(MyPhoToService::class.java)
            val respone = connect.getPhoto()
            if(respone!=null)
                respone.forEach{
                    Log.d("---response----", "${it.id} - ${it.image}")
                    imageView.load(it.image) {
                        placeholder(R.drawable.ic_launcher_foreground) //
hình ảnh tạm khi đang tải
                        error(R.drawable.ic_launcher_background) // hình
ảnh hiển thị khi có lỗi
                    }
                }
        }

    }
}
```

Thực hành: ứng dụng download ảnh sao hỏa



Kết luận

- Tổng kết:
  - Kết nối mạng là phần quan trọng trong các ứng dụng hiện đại.
  - Retrofit là lựa chọn tối ưu cho các dự án sử dụng RESTful API.
- Khuyến nghị:
  - Sử dụng HttpURLConnection cho các tác vụ đơn giản.
  - Sử dụng Retrofit cho các dự án lớn và phức tạp.

Bài tập thực hành

- Viết một ứng dụng Android đơn giản sử dụng **HttpURLConnection** để lấy dữ liệu từ một API công khai.
- Triển khai một ứng dụng sử dụng **Retrofit** để tải danh sách người dùng từ API giả lập.
- Tạo một ứng dụng với **Firebase Realtime Database**, thêm và đọc dữ liệu từ database.

Câu hỏi thảo luận

- Tại sao kết nối mạng lại quan trọng trong lập trình di động?
- RESTful API có những ưu và nhược điểm gì so với SOAP?
- Khi nào nên sử dụng Retrofit thay vì HttpURLConnection?
- Làm thế nào để Firebase hỗ trợ việc đồng bộ dữ liệu thời gian thực?

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Thank you!

THÁI NGUYÊN, THÁNG 10 NĂM 2024